BÀI TẬP & BÀI THỰC HÀNH

NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 1: Giới thiệu

5 bài tập tự luận + 3 bài thực hành lập trình/Linux

PHẦN 1: BÀI TẬP

Bài tập 1

Một máy tính có kiến trúc CPU với thanh ghi 64 bit và không gian địa chỉ bộ nhớ 64 bit. Máy tính này được trang bị một ổ đĩa (thiết bị lưu trữ không khả biến) có dung lượng danh nghĩa **2 TB**, tốc độ truyền dữ liệu tuần tự là **500 MB/s**.

a) Từ (word) của kiến trúc này gồm bao nhiêu byte?

b) Tính dung lượng chính xác của ổ đĩa 2 TB nói trên theo đơn vị byte (dùng quy ước 1 KB = 1.024 byte, tức các lũy thừa của 1.024 như đã học trong chương).

c) Giả sử toàn bộ dữ liệu trên ổ đĩa được đọc tuần tự với tốc độ không đổi 500 MB/s (1 MB = 1.024^2 byte), hãy tính thời gian (giây) cần thiết để đọc hết toàn bộ ổ đĩa, sau đó quy đổi ra phút.

Bài tập 2

Một hệ thống tệp quản lý không gian trống trên đĩa bằng **bitmap**, trong đó mỗi khối (block) trên đĩa được biểu diễn bằng đúng 1 bit (1 = đã dùng, 0 = còn trống). Đĩa có tổng cộng **8.388.608 khối**, trong đó hiện có **3.000.000 khối** đang được sử dụng.

a) Tính số byte cần thiết để lưu toàn bộ bitmap này.

b) Tính số khối còn trống trên đĩa.

c) Giả sử thay vì dùng bitmap, hệ thống dùng một **danh sách liên kết (linked list)** để quản lý các khối trống — mỗi khối trống ứng với một node gồm 8 byte lưu con trỏ tới node kế tiếp và 4 byte lưu số hiệu khối (tổng 12 byte/khối trống). Với số khối trống tính được ở câu b, hãy tính dung lượng bộ nhớ cần dùng cho phương án danh sách liên kết này, so sánh với dung lượng bitmap ở câu a, và kết luận phương án nào tiết kiệm không gian hơn trong tình huống này.

Bài tập 3

Nhân hệ điều hành cần lưu trữ thông tin của **n = 1024** tiến trình (process) trong một cấu trúc dữ liệu để tìm kiếm theo mã tiến trình (PID). Giả sử mỗi phép so sánh khi tìm kiếm tốn trung bình **2 nano-giây (ns)**.

a) Nếu dữ liệu được tổ chức bằng **cây tìm kiếm nhị phân (BST)** nhưng bị suy biến hoàn toàn thành một danh sách liên kết (trường hợp xấu nhất — chèn dữ liệu theo thứ tự đã sắp), hãy tính số phép so sánh tối đa cần thực hiện để tìm một phần tử, biết độ phức tạp trong trường hợp này là O(n).

b) Nếu dữ liệu được tổ chức bằng một **cây tìm kiếm nhị phân cân bằng**, độ phức tạp tìm kiếm là O(lg n). Hãy tính số phép so sánh tối đa cần thiết (lấy log cơ số 2 của n).

c) Tính thời gian tìm kiếm tối đa (theo ns) tương ứng với câu a) và câu b), sau đó tính tỉ lệ chênh lệch (số lần nhanh hơn) giữa cây cân bằng so với cây suy biến.

Bài tập 4

Một hệ thống cần truyền một tệp dữ liệu dung lượng **4 MB** (tính theo 4 × 1.024 × 1.024 byte) từ đĩa vào bộ nhớ chính. Giả sử chi phí xử lý mỗi ngắt (lưu ngữ cảnh CPU, xác định loại ngắt, gọi thường trình phục vụ ngắt, khôi phục ngữ cảnh) tốn trung bình **2 micro-giây (µs)**.

a) Nếu hệ thống **không dùng DMA**, mỗi byte truyền xong phát sinh một ngắt riêng, hãy tính tổng số ngắt phát sinh cho toàn bộ tệp, và tổng thời gian hệ thống tiêu tốn chỉ riêng cho việc xử lý các ngắt đó.

b) Nếu hệ thống dùng **DMA (Direct Memory Access)**, truyền dữ liệu theo từng khối **4.096 byte** và chỉ phát sinh 1 ngắt cho mỗi khối, hãy tính số ngắt cần thiết và tổng thời gian xử lý ngắt tương ứng.

c) Tính số lần tổng thời gian xử lý ngắt giảm được khi chuyển từ phương án a) sang phương án b).

Bài tập 5

Một hệ thống máy tính đơn xử lý (uniprocessor) xử lý được trung bình **200 tiến trình/giây**. Nhà quản trị nâng cấp hệ thống lên kiến trúc **đa xử lý đối xứng (Symmetric Multiprocessing – SMP)** với **6 CPU** giống hệt nhau, tải công việc được chia đều cho các CPU.

a) Giả sử không có bất kỳ chi phí đồng bộ hóa nào (trường hợp lý tưởng), hãy tính thông lượng lý thuyết (số tiến trình/giây) của hệ thống SMP 6 CPU này.

b) Trong thực tế, do chi phí đồng bộ hóa giữa các CPU (tranh chấp bus chung, duy trì tính nhất quán bộ nhớ đệm — cache coherency), hệ thống chỉ đạt được **85%** thông lượng lý thuyết tính ở câu a). Hãy tính thông lượng thực tế của hệ thống.

c) Từ kết quả câu b), tính hiệu suất trung bình mà mỗi CPU trong hệ SMP đóng góp được (số tiến trình/giây/CPU), so sánh với hiệu suất 200 tiến trình/giây của 1 CPU đơn lẻ ban đầu, và nêu nhận xét về việc "tính kinh tế theo quy mô" (một trong các ưu điểm của đa xử lý đã học) bị suy giảm như thế nào khi số CPU tăng lên.

PHẦN 2: BÀI THỰC HÀNH

Bài thực hành 1: Khám phá /proc và quan sát tiến trình hệ thống

Mục tiêu: Hiểu cách hệ điều hành Linux phơi bày thông tin về tiến trình (process) và trạng thái hệ thống thông qua pseudo-filesystem `/proc`, liên hệ với nội dung "Quản lý tiến trình" và "Hoạt động của hệ thống máy tính" đã học trong chương.

Yêu cầu: Môi trường Linux + bash; các lệnh `ps`, `top`/`uptime`, `grep`, `find`, `wc`; không cần biên dịch chương trình C.

Đề bài chi tiết: Viết một script bash tên `explore_proc.sh` thực hiện:

1. In ra PID của chính script và một số trường thông tin của chính nó đọc trực tiếp từ `/proc/self/status` (Name, State, Pid, PPid, VmRSS, Threads).
2. In ra nội dung `/proc/self/stat` (một dòng số liệu thô do nhân cung cấp).
3. In ra load average của hệ thống bằng lệnh `uptime`.
4. Liệt kê 5 tiến trình đang dùng nhiều CPU nhất bằng `ps`.
5. Đếm tổng số tiến trình đang chạy trên hệ thống bằng cách đếm số thư mục có tên là số nguyên trực tiếp trong `/proc`, rồi đối chiếu với số tiến trình mà `ps -e` báo cáo.

Các bước thực hiện:

1. Tạo file `explore_proc.sh`, gõ nội dung script (xem lời giải Phần 4).
2. Cấp quyền thực thi: `chmod +x explore_proc.sh`.
3. Chạy thử: `./explore_proc.sh` và quan sát output.
4. So sánh số liệu ở bước 5 (đếm qua `/proc`) với số liệu `ps -e` báo cáo — giải thích nếu có chênh lệch nhỏ (do tiến trình sinh/kết thúc trong lúc đo).

Bài thực hành 2: Đo overhead lời gọi hệ thống (system call) so với lời gọi hàm thư viện thuần user-mode

Mục tiêu: Minh họa bằng số liệu thực đo sự khác biệt về chi phí thời gian giữa một lời gọi hàm hoàn toàn chạy trong chế độ người dùng (user mode) và một lời gọi hệ thống thực sự (system call) buộc CPU phải chuyển sang chế độ nhân (kernel mode) rồi quay lại — đúng với nội dung "Hoạt động hai chế độ (dual-mode)" đã học.

Yêu cầu: Linux + gcc (C11); header `time.h`, `unistd.h`, `sys/syscall.h`.

Đề bài chi tiết: Viết chương trình C `mode_switch_bench.c`:

- Gọi một hàm thuần user-mode (không có bất kỳ system call nào bên trong, ví dụ một phép tính số học đơn giản) **1.000.000 lần**, đo tổng thời gian bằng `clock_gettime(CLOCK_MONOTONIC, ...)`.

- Gọi trực tiếp system call `getpid()` thông qua `syscall(SYS_getpid)` (buộc chuyển mode thật sự, không qua cache của glibc) **1.000.000 lần**, đo tổng thời gian tương tự.

- In ra tổng thời gian, thời gian trung bình mỗi lần gọi (ns/lần) của từng loại, và tỉ lệ chênh lệch giữa hai loại.

Các bước thực hiện:

1. Tạo file `mode_switch_bench.c` với nội dung ở Phần 4.
2. Biên dịch: `gcc -Wall -O2 -o mode_switch_bench mode_switch_bench.c`.
3. Chạy: `./mode_switch_bench` và ghi lại 2 giá trị trung bình ns/lần.
4. Chạy strace đối chiếu để xác nhận số lượng system call thực sự phát sinh: `strace -c -e trace=getpid ./mode_switch_bench` — đếm số lần `getpid` xuất hiện trong bảng thống kê của strace và so với 1.000.000 lần gọi trong code.
5. Nhận xét: vì sao lời gọi qua `syscall(SYS_getpid)` luôn chậm hơn hàm user-mode thuần túy, liên hệ tới chi phí chuyển đổi bit chế độ (mode bit) mà phần cứng phải thực hiện.

Bài thực hành 3: Dùng `strace`/`time` đo số lượng system call và overhead vào/ra theo byte so với theo khối

Mục tiêu: Quan sát trực tiếp số lượng system call (read/write) một chương trình thực sự phát sinh và thời gian thực thi tương ứng, liên hệ với khái niệm "một ngắt cho mỗi khối thay vì mỗi byte" (DMA) đã học trong Bài tập 4.

Yêu cầu: Linux + gcc (C11); các lệnh `strace`, `time`; header `fcntl.h`, `unistd.h`.

Đề bài chi tiết: Viết chương trình C `io_test.c` nhận tham số dòng lệnh:

- `create`: tạo một file `testfile.bin` dung lượng 10 MB.
- `byte`: mở file và đọc **từng byte một** bằng nhiều lời gọi `read(fd, &c, 1)`.
- `block`: mở file và đọc theo **khối 4096 byte** mỗi lần gọi `read()`.

Sau đó dùng `strace -c` để đếm số lượng system call `read` phát sinh trong mỗi cách đọc, và dùng `time` để đo thời gian thực thi (real/user/sys), từ đó so sánh chi phí giữa đọc theo byte và đọc theo khối.

Các bước thực hiện:

1. Tạo file `io_test.c` với nội dung ở Phần 4.
2. Biên dịch: `gcc -Wall -O0 -o io_test io_test.c`.

3. Tạo file kiểm thử: `./io_test create`.

4. Đếm system call khi đọc từng byte: `strace -c ./io_test byte` — ghi lại số lần gọi `read`.

5. Đếm system call khi đọc theo khối: `strace -c ./io_test block` — ghi lại số lần gọi `read`.

6. Đo thời gian thực thi: `time ./io_test byte` và `time ./io_test block`, so sánh cột `real` và `sys`.

7. Tính tỉ lệ số lần gọi `read` giữa hai cách (phải xấp xỉ 4096 lần, đúng bằng kích thước khối), đối chiếu với kết quả tính tay ở Bài tập 4.

========== END ==========